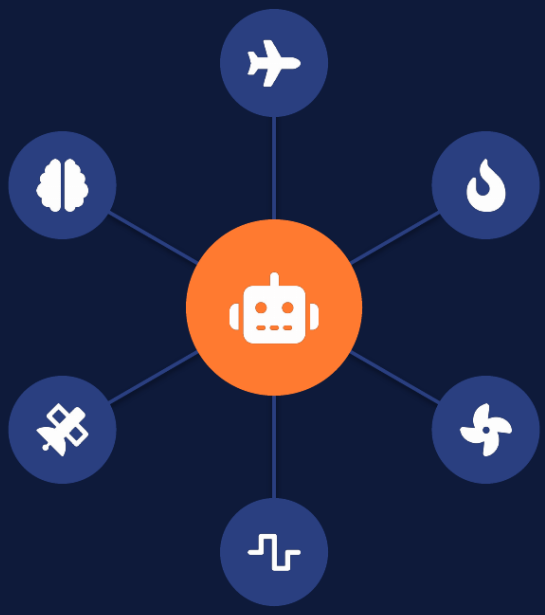
MONTHLY SHOW-AND-TELL · ENGINEERING

Automating Your Engineering Workflow with AI

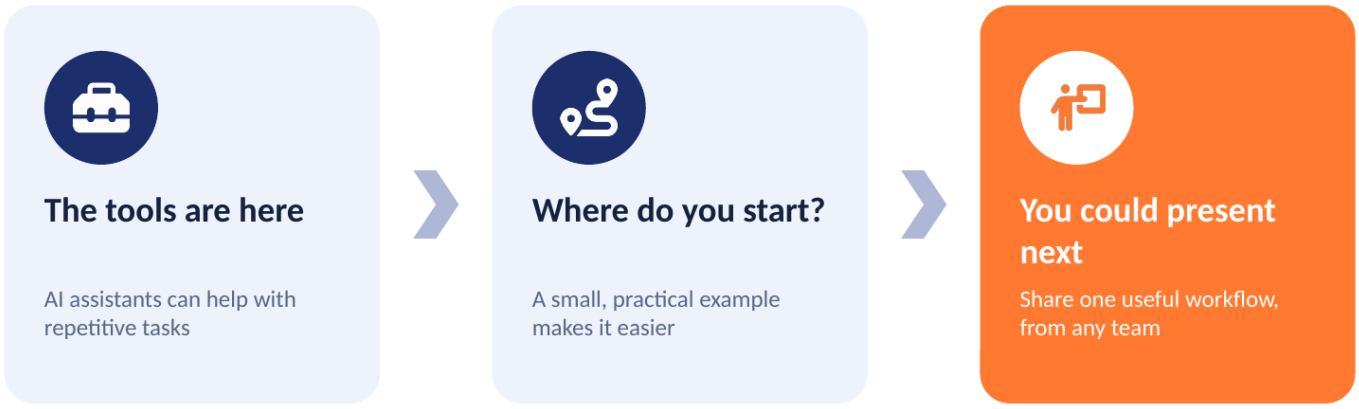
One workflow at a time. No experts required.

Kalki Sharma · Computational Aerodynamics



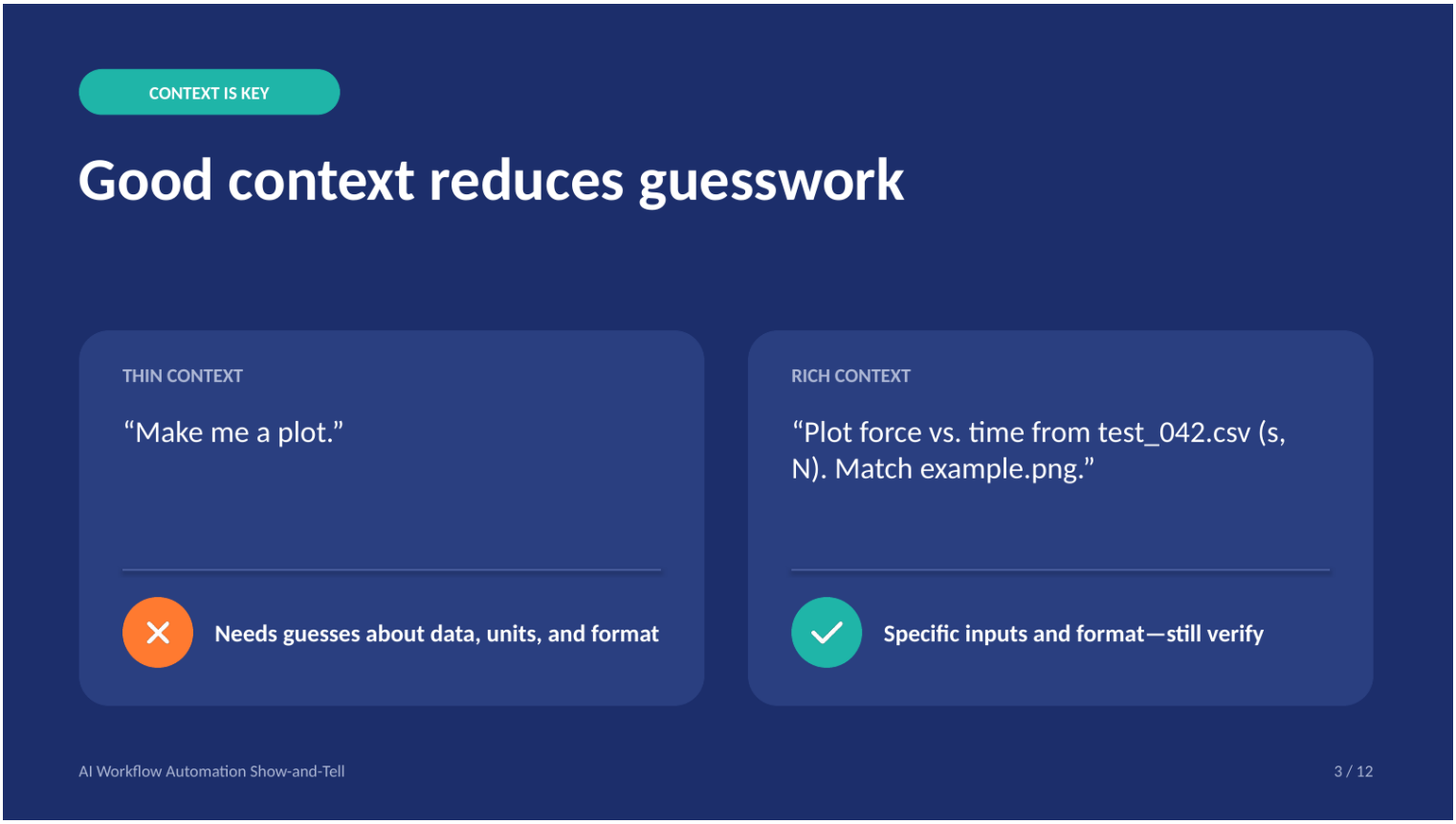
SLIDE 1 OF 12 · SPEAKER NOTES

Welcome to our monthly, informal show-and-tell on AI-assisted engineering workflows. No AI expertise is assumed. Whether your work involves design, test, analysis, software, or project coordination, you probably have a repetitive task worth exploring. We'll cover a few concepts, see practical examples, and keep the emphasis on workflows you could try yourself.



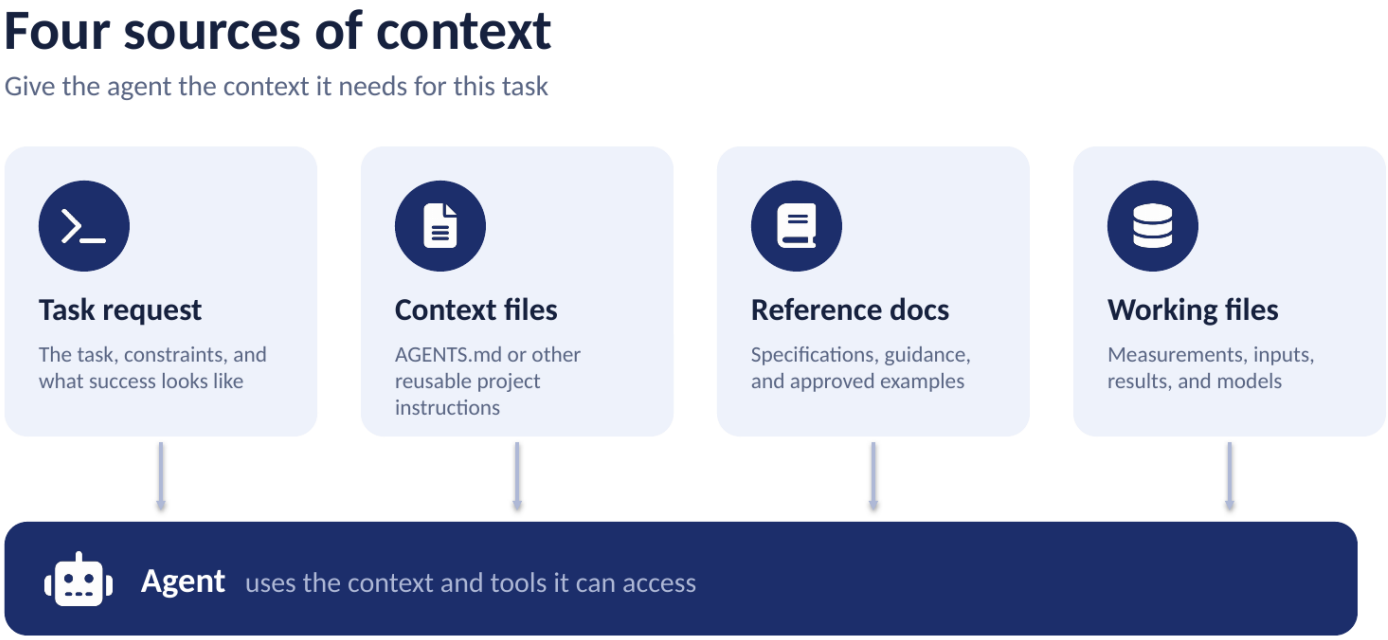
SLIDE 2 OF 12 · SPEAKER NOTES

The goal isn't to turn everyone into an AI developer. An agent is simply an AI assistant that can use permitted tools and files to work through steps. The tools can be useful, but the starting point isn't always obvious. This series gives us a place to swap small, practical examples. Eventually, I'd love to see people from different engineering teams share what worked, including the mistakes and lessons.



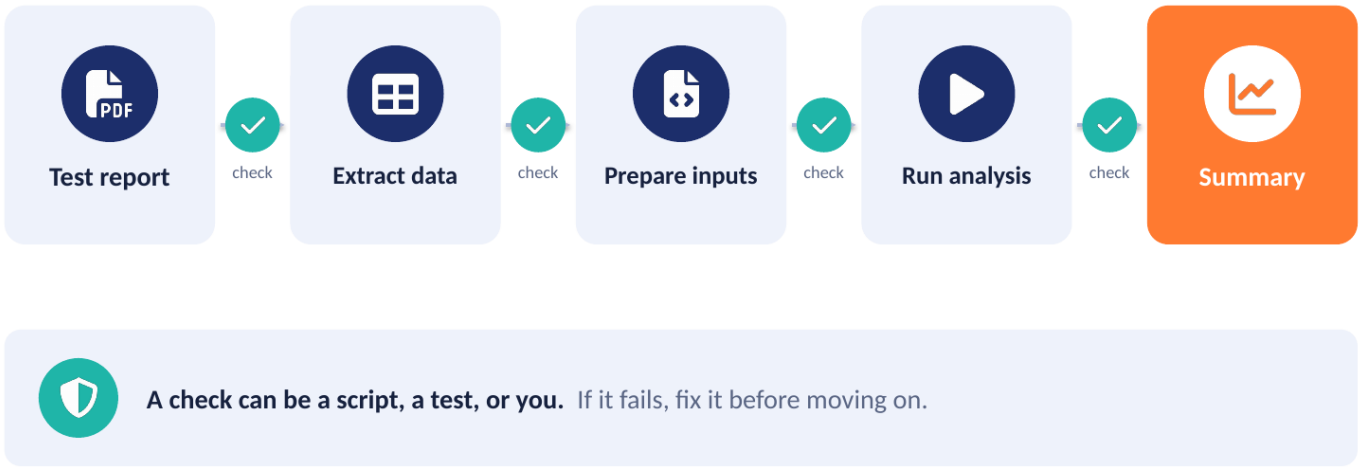
SLIDE 3 OF 12 · SPEAKER NOTES

Consider the same plotting request. 'Make me a plot' leaves the data source, quantities, units, and output style ambiguous. A more specific request names the file, units, and a reference figure. That reduces guesswork, but does not guarantee a correct plot. We still need to examine the inputs and check the result. Context improves the odds of getting something useful on the first pass.



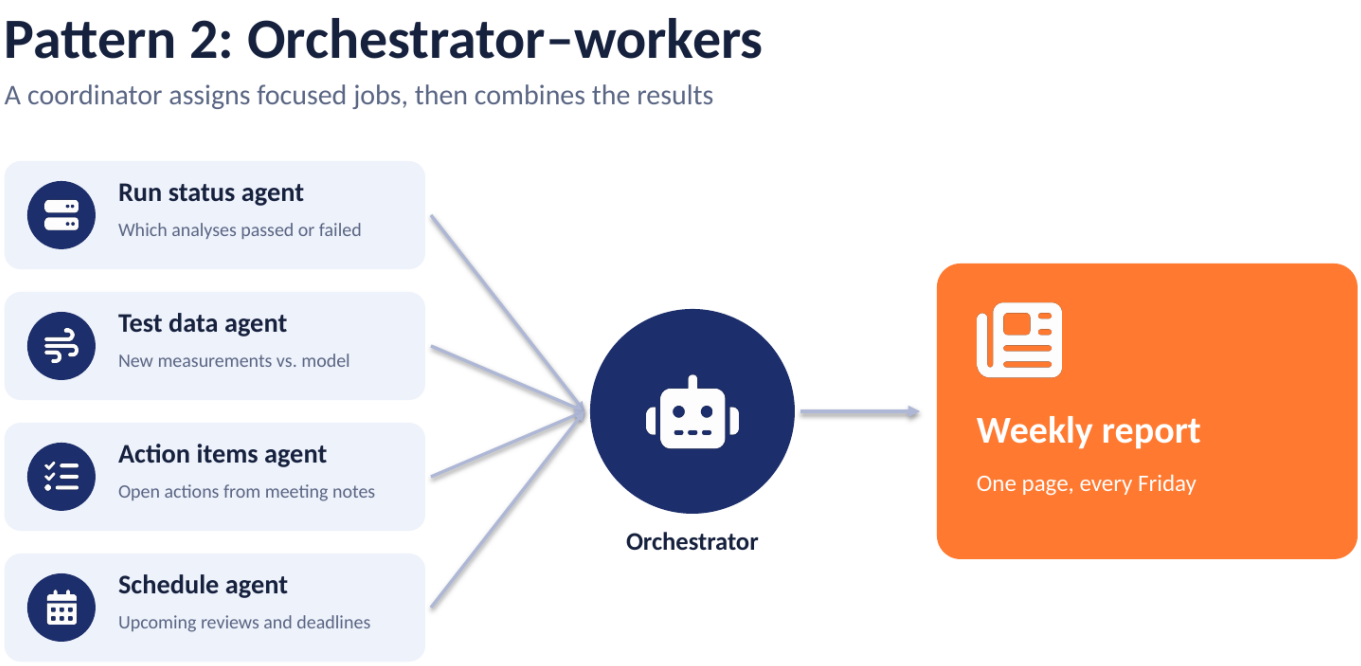
SLIDE 4 OF 12 · SPEAKER NOTES

Think of four sources of context. First, the current task and definition of success. Second, reusable project instructions, such as AGENTS.md if the tool supports them. Third, reference material: manuals, specifications, and examples. Fourth, the working files and data. You don't always need all four. The important point is that the agent can only use material and tools it is actually authorized and able to access.



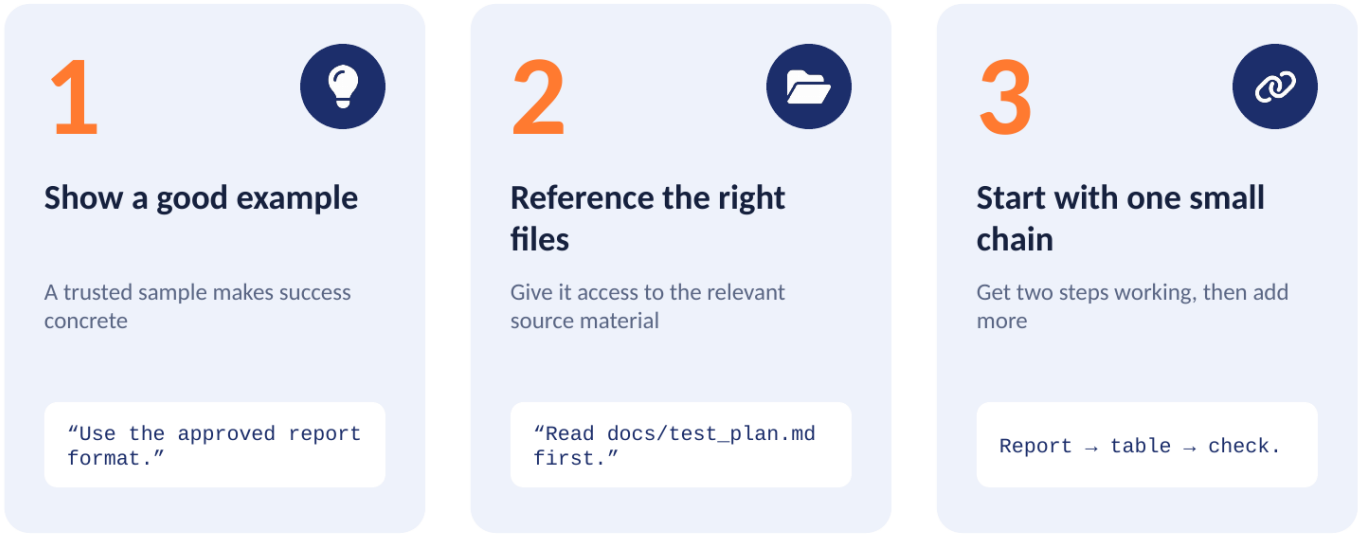
SLIDE 5 OF 12 · SPEAKER NOTES

The first pattern is a chain: one step produces something the next step needs. For example, read a test report, extract values, prepare model inputs, run an analysis, and summarize the results. Put a check at each handoff: missing values, wrong units, invalid inputs, failed jobs, or implausible outputs. A check can be code or a human. Don't automate a long chain until the small pieces behave reliably.



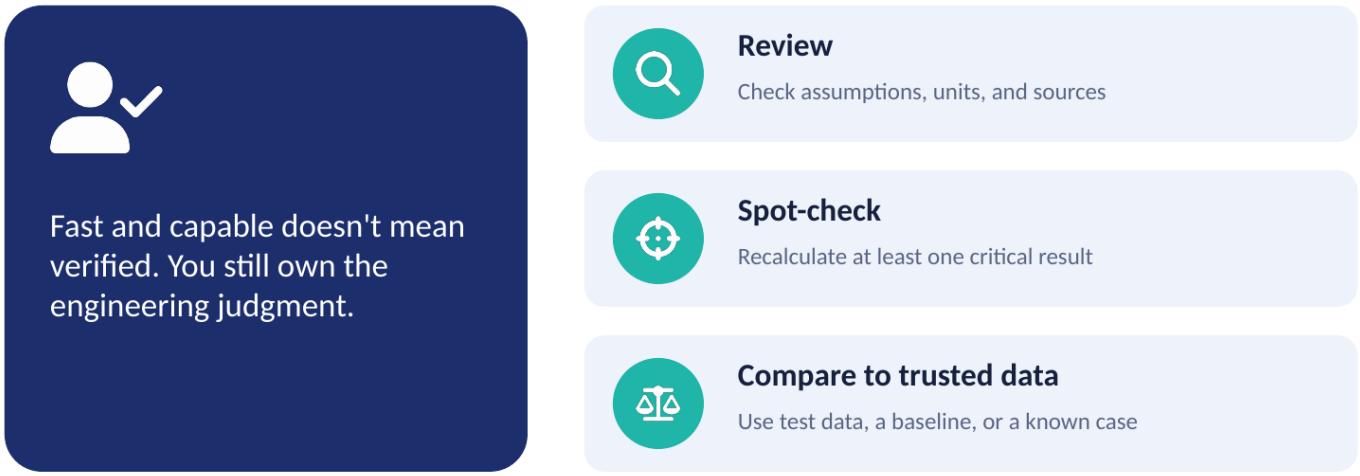
SLIDE 6 OF 12 · SPEAKER NOTES

The second pattern works when jobs can be split. A coordinator asks different workers for run status, measurements, action items, and milestones, then combines their findings in a weekly report. A worker might be a separate agent, script, or tool call. Unlike a chain, these tasks need not depend on one another. This is useful for larger workflows, but it's not the place to start if one simple chain will do.



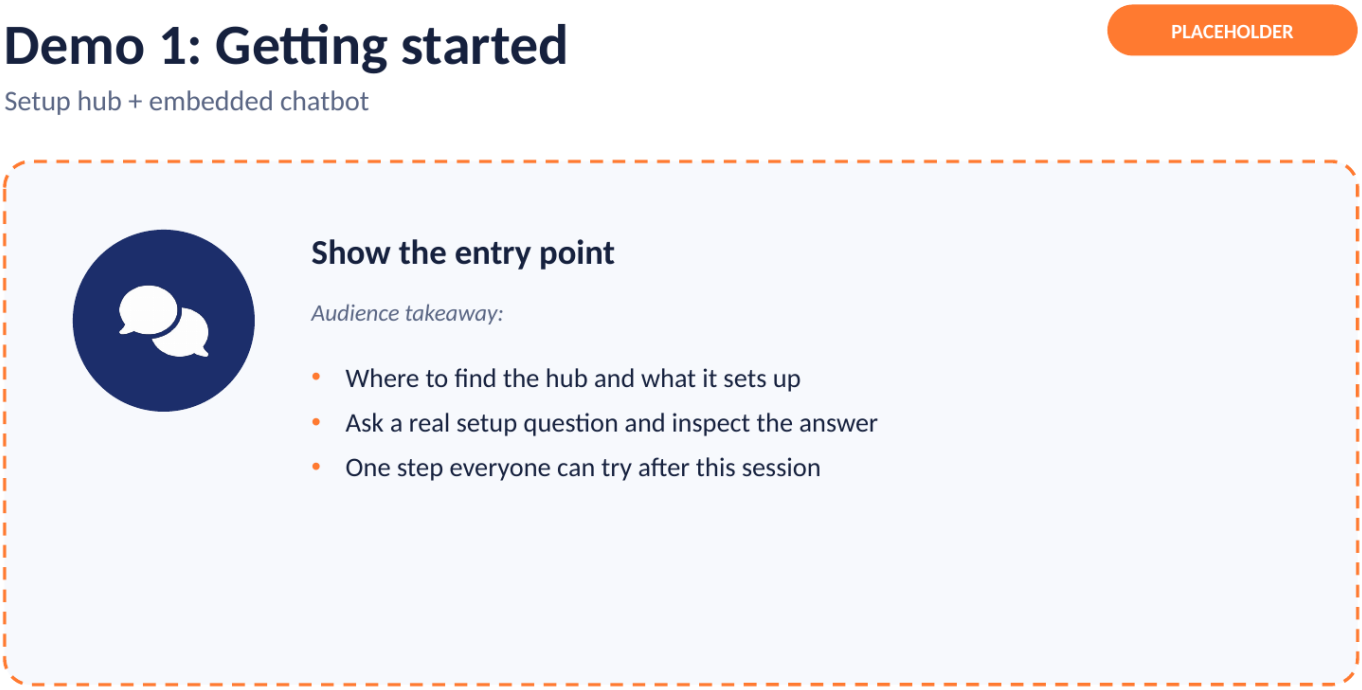
SLIDE 7 OF 12 · SPEAKER NOTES

Three habits that help. Show a good example of the output you want, while still supplying any important requirements. Give the assistant access to the specific files it needs rather than copying everything into a giant prompt; follow the data-handling rules for your tools. Finally, start with two reliable steps before growing the workflow. The aim is a repeatable process, not the most elaborate collection of agents.



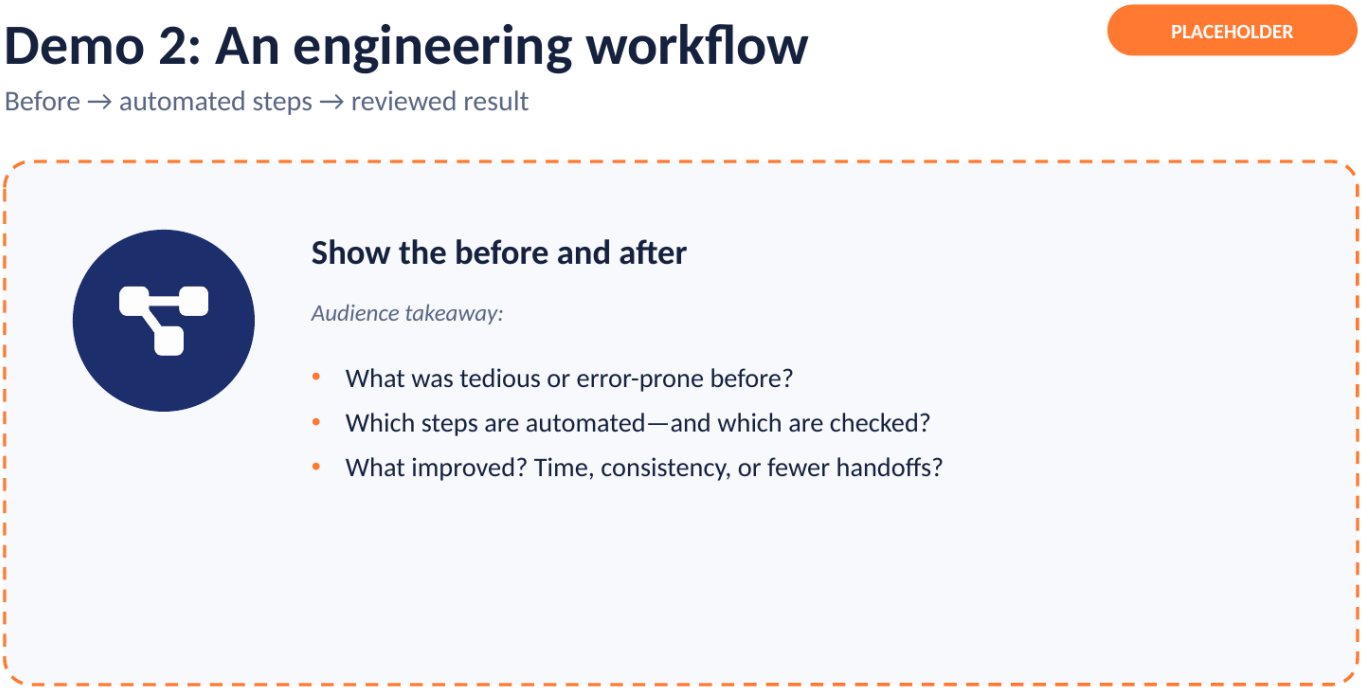
SLIDE 8 OF 12 · SPEAKER NOTES

This is the engineering question: how do we know the output is right? Review the method and inspect assumptions, units, and sources. Recalculate a meaningful result independently. Compare it with test data, an approved baseline, or a known case. Keep normal quality, technical review, and data-control processes in place. AI can accelerate parts of the work; responsibility for the engineering conclusion remains with us.



SLIDE 9 OF 12 · SPEAKER NOTES

[DEMO CONTENT PENDING] Start with the setup hub. Show exactly where a new user finds it, what setup it handles, and one common question you can ask the embedded chatbot. Inspect the answer rather than simply accepting it. End with a one-step challenge the audience can try after this meeting. Replace the placeholder with a cleared screenshot or live view before presenting.



SLIDE 10 OF 12 · SPEAKER NOTES

[DEMO CONTENT PENDING] Choose a program-cleared example with a recognizably tedious before state. Briefly show the inputs, the automated steps, the verification checkpoint, and the reviewed result. Tie it back to chaining or coordinator-worker patterns without overexplaining implementation. Quantify time saved only if you've measured it. Confirm every screenshot, file name, and result is approved for this audience.

Demo 3: RCAS end to end

PLACEHOLDER

Input → checks → reviewed output



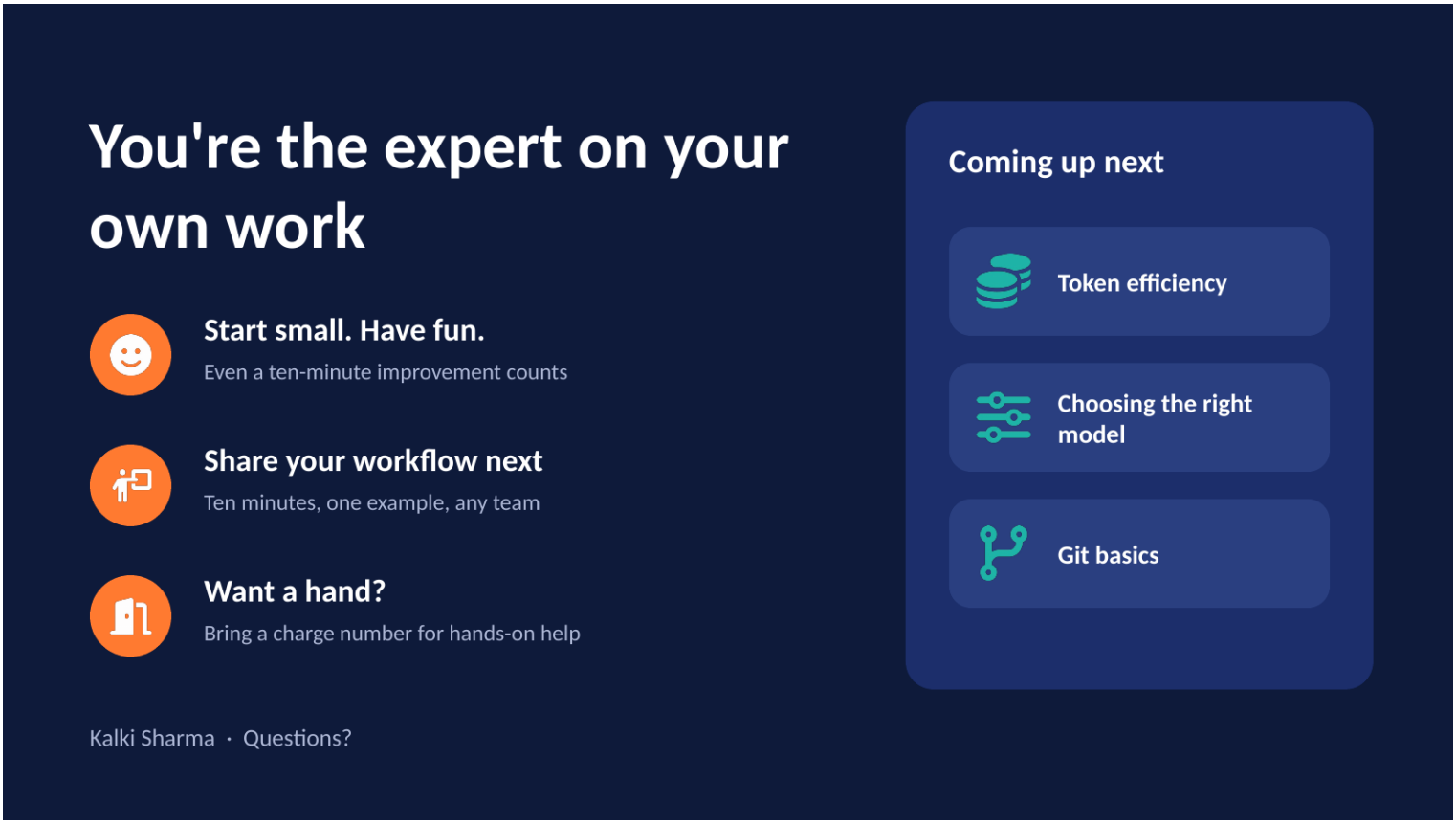
Walk through a real run

Audience takeaway:

- Show the inputs and what the workflow produces
- Highlight one concrete verification gate
- Use a recorded example if the live run is unavailable

SLIDE 11 OF 12 · SPEAKER NOTES

[DEMO CONTENT PENDING] For RCAS, show one cleared input, the sequence of tasks, a real validation point, and the final artifact. Mention how a failed check is handled rather than presenting only a perfect run. A short recording can replace the live portion if access, runtime, or connectivity is uncertain. Keep sensitive program data out of the slide and screen-share unless approved.



SLIDE 12 OF 12 · SPEAKER NOTES

You are the expert in your own work; the agent is a tool you direct and check. Start with one small repetitive task and see whether it helps. A short, imperfect demonstration is welcome next month; the purpose is to learn from each other. If you'd like hands-on help, reach out with a charge number. Future sessions can explore token efficiency, model selection, and Git basics. Questions?